

AEGISPAY

LangGraph Architecture — SELL (AI Buyer Checkout)

How an AI buys a product for a real customer, step by step — **without the AI ever touching the money.**

1. What this graph does (in plain English)

A user says something like: *“Find running shoes under Rs 4,000.”* The SELL agent graph turns that sentence into a **ready-to-pay business intent**. It understands, searches, recommends, builds a cart, and asks for permission. It never swipes a card.

The most important idea

LangGraph is only the brain. The money lives in a separate AegisPay control plane that the graph cannot reach. The graph hands over a validated intent; only the control plane can say yes and move money.

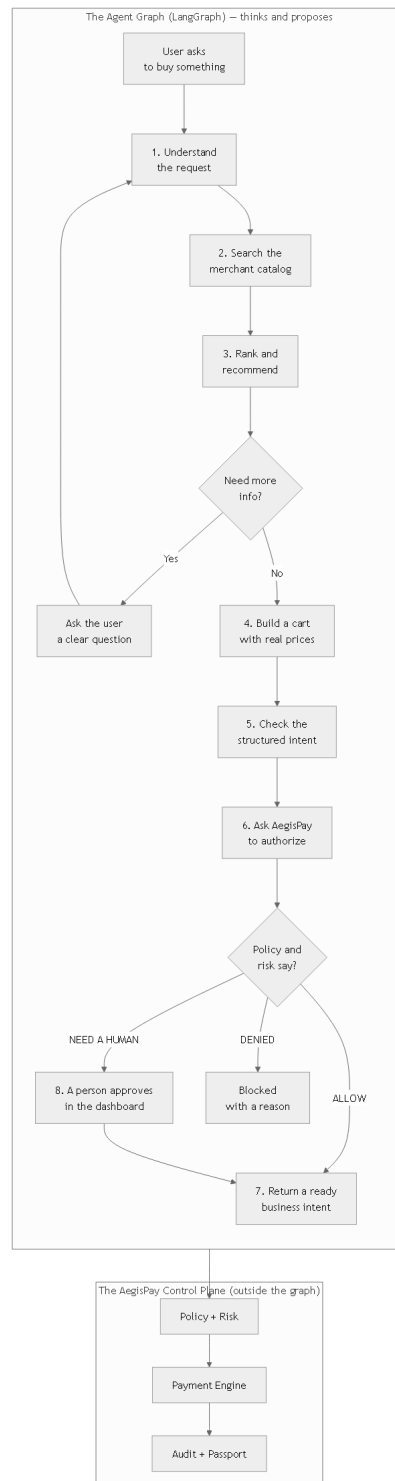


Figure 1 — The SELL agent graph (top) and the money control plane (bottom) it cannot reach

2. What happens at each step

#	Node	What it does	Minder (the safety rail)
1	Understand	Reads the request as a typed intent (product, budget, limits).	Pydantic schema; budget is read to the diagram, not trusted later.
2	Search catalog	Finds matching products from the merchant catalog.	Only typed catalog data; never raw product text as instructions.
3	Rank & recommend	Chooses the best few with a short reason.	Prices and stock come from the server, not from the AI.
4	Clarify	Asks the user a clear question if anything is unclear.	Limited to asking a question; cannot change a price.
5	Build cart	Adds chosen items with server prices; computes a cart hash.	Cart is locked and hashed, so it cannot be silently changed later.
6	Validate intent	Checks the whole plan is structured and complete.	Any malformed or suspicious intent is rejected, not guessed.
7	Authorize	Calls AegisPay to check policy + risk.	AegisPay decides; the AI cannot decide.
8	Human / finish	Low risk = done; high risk = a person approves, then it resumes.	Approval is scoped, expiring, and can't be reused.

3. The state machine (how it pauses for a person)

A purchase is not a one-shot answer. When AegisPay says **human approval** is needed, LangGraph **pauses** in the middle, the person approves in the dashboard, and the same graph **resumes** where it stopped. Nothing is lost or redone.

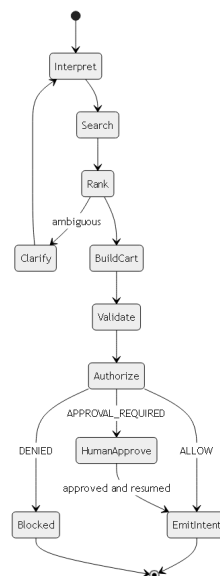


Figure 2 — The graph's internal states, including the human-approval pause

4. The safety boundary (the thing that makes it trustworthy)



Figure 3 — The AI thinks; the control plane acts. Only a structured intent crosses the line.

Only the AI side may...	Only the control plane may...
Understand a request	Run policy and risk checks
Search and recommend	Decide ALLOW / APPROVAL / DENY
Build and lock a cart	Authorize the payment
Ask for a permission	Move money through Razorpay
Produce a structured intent	Write the Passport and audit trail

5. Built to run in production

• **Resumes safely.** Every step is saved (checkpointed) so a pause, crash, or approval can continue exactly where it left off.

• **No double work.** An idempotency key means a retried step does not run twice and cannot create a second cart or intent.

• **It can't invent prices.** Prices, stock and totals are always the server's answer, so the AI cannot change the amount.

• **It can't skip limits.** Every intent passes policy + risk in the control plane, no matter what the AI says.

• **You can watch it.** The graph logs its trace, and every decision is written to the audit ledger and shown as a Transaction Passport.

• **It fails safely.** If the AI is slow, wrong, or unavailable, the control plane simply keeps the money safe and waits.

• **It's tested.** Unit tests for every node, plus a red-team that tries to cheat the graph (prompt injection, price tampering, fake approval) and must fail.

6. What you get

A user can buy through an AI safely and confidently. The merchant sells more (SELL). The customer and the merchant have full proof of every decision. And the AI was never in charge of money.